sensors

MDPI

Article

利用 PMR-Dueling DQN 算法增强移动机器人路径规划的稳定性和性能

Demelash Abiye Deguale [1,*]、Lingli Yu [1]、Melikamu Liyih Sinishaw [2] 和 Keyi Li [1]

1 中南大学自动化学院, 长沙 410083; llyu@csu.edu.cn (L.Y.); 214612147@csu.edu.cn (K.L.) 2 中南大学计算机科学与工程学院, 长沙 410083; melikamuliyih29@gmail.com * 通讯：deguale97@gmail.com

摘要：复杂环境下移动机器人的路径规划仍然是一个具有挑战性的问题。这项工作引入了一种改进的机器人导航深度强化学习策略，该策略结合了决斗架构、优先体验重放和形状奖励。在网格世界和两个具有静态和动态障碍物的 Gazebo 模拟环境中，将具有修改奖励和优先体验重放的决斗深度 Q 网络 (PMR-Dueling DQN) 算法与 Q 学习、DQN 和 DDQN 在路径最优性、碰撞避免和学习速度方面进行比较。为了鼓励最佳路线，成形的奖励函数考虑了目标方向、避障和距离。优先重播将训练集中在重要事件上，而决斗架构则将价值学习和优势学习分开。结果表明，PMR-Dueling DQN 极大地提高了跨条件下的收敛速度、稳定性和整体性能。在网格世界和 Gazebo 环境中，PMR-Dueling DQN 都获得了更高的累积奖励。深度强化学习与奖励设计、网络架构和经验回放的结合使 PMR-Dueling DQN 超越了复杂环境中机器人路径规划的传统方法。

关键词：路径规划；决斗网络；移动机器人；优先体验回放；强化学习

check for updates

引文：Deguale，D.A.；于L.；西尼肖，M.L.； Li, K. 使用 PMR-Dueling DQN 算法增强移动机器人路径规划的稳定性和性能。 *Sensors* 2024，*24*，1523。https://doi.org/10.3390/s24051523

学术编辑：Sergio Toral Marín

收稿日期: 2023年12月22日
修订日期: 2024年2月1日 接受日期: 2024年2月7日 发布日期: 2024年2月27日

Copyright: © 2024 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license (https://creativecommons.org/licenses/by/4.0/).

一、简介

近几十年来，移动机器人已广泛应用于军事、工业和安全等各个领域，以执行重要的无人任务。在复杂环境中实现移动机器人有效导航的一个关键挑战是路径规划[1]。路径规划的主要目标是确定机器人初始位置和所需最终位置之间的最佳轨迹。虽然通常有多个可行路径可供选择，但最佳路径是根据最小距离、路径质量、能源效率或最常见的最短距离和时间等标准来选择的[2]。高性能路径规划技术的发展对于移动机器人在障碍物中确定最佳路线并避免不必要的弯路至关重要。

在移动机器人路径规划领域，有效的算法通过考虑路径长度和遍历时间等因素来优化点对点路线，发挥着至关重要的作用。近年来，深度强化学习（DRL）算法的应用和人工智能在该领域的广泛采用推动了显着的转变。这种转变催生了移动机器人的智能、灵活和动态路径规划过程。人工智能和深度强化学习的结合带来了多种优势，包括路径规划的自动化、互联信息物理系统的建立以及云计算促进的实时数据的持续交换。这种集成促进了机器的发展

基于学习的控制算法，提高了移动机器人路径规划在制造过程等不同应用中的能力[3]。

路径规划一般分为两类：局部路径规划和全局路径规划[4]。全局路径规划需要提前充分了解环境，以便在执行之前预先计算连接起点和目的地的无碰撞路径。预先需要完整的环境信息。因此，全局路径规划的主要目标是创建环境并随后优化路径[5]。相反，本地路径规划可以处理未知或部分已知的环境。局部技术也称为反应性方法，用于在存在很大不确定性的陌生空间中导航。局部路径规划需要机器人在整个过程中从周围环境中收集数据和信息，这需要机器人具备学习能力。本地规划者根据传感器反馈实时动态做出反应，而不是依赖预先绘制的路线。在实际应用中，通常采用结合局部和全局规划方法的混合方法，因为当采用这些方法时，在动态和复杂的环境中确保全局路径规划的可靠性变得具有挑战性。考虑到更广泛的环境，全局规划的更新频率通常较低，而局部规划的更新频率更高，以响应智能体采取的每个新行动。一个著名的例子是机器人操作系统（ROS）中的 Move Base 功能包。这种混合策略可以利用全球和本地规划技术的优势，实现高效且适应性强的导航。

近年来，强化学习的快速发展为解决移动机器人路径规划的长期挑战创造了新的潜力[6]。随着强化学习理论和实用算法的不断快速发展，将现代强化学习技术应用于机器人路径规划问题已成为应对这些挑战的一个活跃且有前景的研究领域。这是因为强化学习不需要复杂环境模型的先验知识，而是使代理能够通过与周围环境的直接交互和奖励信号形式的反馈来学习胜任行为。这使得强化学习成为机器人路径规划应用中极具吸引力的方法。强化学习已成功应用于各个领域，包括自主车辆控制[7]、先进机器人技术[8,9]，以及最重要的移动机器人路径规划[10]。在强化学习中，智能体通过与环境交互并观察相关的观点和奖励结果，纯粹从经验中学习最佳行为，最终收敛于允许最大奖励性能的学习价值函数[11]。一个关键优势是强化学习代理可以在状态空间中很少访问的部分获得技能，而这些部分可能缺乏人类提供的示例[12]。然而，虽然传统的 Q 学习算法在路径规划方面表现出了良好的前景，但其收敛速度慢阻碍了解决大规模、复杂的现实问题的有效性[13]。为了解决这个问题，在这项工作中，我们使用了[14]引入的决斗深度 Q 网络以及成形奖励和优先体验重放（PMR-决斗 DQN）方法。通过结合考虑目标接近度、轨迹平滑和每一步避障的成形奖励函数，该方法超越了传统的反馈信号。通过严格设计和适应每个环境方面的结果，精细的训练信号大大增强了算法在困难导航环境中的性能。我们对网格地图环境中的全局路径规划以及 Gazebo 模拟环境中的全局和局部路径规划的标准 Q 学习、DQN 和 DDQN 方法进行了比较分析。我们对网格世界和带有移动代理的 Gazebo 模拟环境的评估表明，PMR-Dueling DQN 算法在累积的平均奖励、采取的步数和遍历的路径长度方面优于其他算法。

总的来说，本文有以下主要贡献上：

1、采用PMR-Dueling DQN方法进行复杂动态情况下移动机器人路径规划，解决DQN算法不稳定、收敛速度慢的问题。为了在收敛速度、稳定性和路径规划性能方面实现卓越的性能，该算法集成了深度神经网络、决斗架构、优先体验回放和形状奖励。

2.除了传统的网格地图外，还使用了更真实的Gazebo模拟设置。名为 Gazebo 的物理模拟平台通过提供与真实环境非常相似的简化环境模型，弥合了真实环境和虚拟环境之间的差距。这种方法保证了代理学习的方法适用于实际的机器人环境。

3.为了解决障碍物的影响，为每个环境模型建立了一个成形的奖励函数，通过结合避障和到目标位置的距离度量，有助于增强规划路径并提高算法的收敛速度。这一改进旨在在避开障碍物和朝着目标前进之间取得平衡，使机器人能够设计一条避开障碍物区域的路径，从而解决与稀疏奖励问题相关的挑战。 4.进行比较分析，通过与传统Q学习和其他深度强化学习（DRL）算法进行比较来评估我们提出的方法的效率。

本文的其余部分组织如下。在第2-4节中，我们分别介绍相关工作、理论基础和方法。我们在第 5 节中进行了模拟实验并分析了结果。在最后一节，即第 6 节中，我们总结了本文的工作。

2.相关工作

多年来，已经开发了各种传统的全球规划技术。 Dijkstra 的方法 [15] 选择最接近起始点的节点并产生最短路径，但与 BFS 和 A* [16] 相比，计算成本较高。概率路线图（PRM）方法[17]利用图搜索来离散化 A* 搜索等算法的连续空间。 PRM 因其有效查找短路径的能力而被广泛使用。快速探索随机树（RRT）方法[18]可以构建实时路径并很好地处理高维空间，但与 Dijkstra 或 PRM 相比，可能实现较差的路径最优性。

强化学习最近在机器人规划方面取得了重大进展。参考号[19] 为基于网格的任务开发了多个 Q 网络，而 [20] 改进了用于网格路径规划的 Q 学习收敛。然而，随着环境的发展，Q 学习也存在局限性，需要长时间计算 Q 值更新和更大的内存矩阵。学习矩阵也可能不是最优的。为了解决这个问题，参考文献。 [21]将元启发式方法集成到 Q 学习中以减少最优解搜索时间。参考号[22]提出了一种启发式强化学习方法，通过结合成本函数和启发式方法来选择策略。添加成本学习可以平衡强化学习中的奖励和成本，并在机器人路径规划问题上得到验证。后向 Q 学习 [23] 结合了 Q 学习和 Sarsa，以便在各种系统中更快地收敛。参考号[24]结合Q学习和神经网络来有效地处理机器人导航的动态和静态障碍物。通过预设速度，他们的技术简化了时间关键型应用。评估显示出积极的结果。参考号[25]将 Q 学习和差分进化集成到模因算法中，相对于 GA、PSO 和差分进化，增强了多机器人导航性能。

最近，使用深度强化学习（DRL）算法作为基本控制[26]、导航[27]、定位[28]和规划[29]系统已经流行起来。人工神经网络的灵活性，以及模拟、训练和利用它们作为端到端解决方案的能力，引起了研究界的兴趣。这促使人们努力将 DRL 的这些进步集成到机器人应用中。神经网络的适应性与训练能力相结合

通过仿真，它们非常适合作为各种机器人功能的综合系统。强化学习决策和神经网络拟合相结合，用于特定环境中的规划[30]。然而，这些方法存在复杂空间中的维数问题、收敛速度慢和缺乏可移植性等局限性[31]。为了解决这些问题，参考文献。 [32]介绍了针对核环境量身定制的改进决斗深度双Q网络（ID3QN）算法。该算法利用非对称神经网络结构和优先经验重播机制，解决了传统 DQN 算法中普遍存在的高估问题。然而，虽然在放射性网格场景中表现出色，但它可能缺乏对各种复杂环境的适应性，并且可能不会优先考虑实时避障。在[33]中，一种将强化学习与深度Q网络（DQN）相结合的新算法旨在使代理能够在Gazebo模拟中进行实时决策和轨迹规划。然而，这种方法在促进探索的同时，可能会忽视轨迹规划过程中的实时避障，从而可能限制其在动态环境中的有效性。

为了解决 DQN 中的高估问题，人们提出了各种改进算法，例如[34]的 Double DQN (DDQN)，它使用两个网络来估计动作的值以减少高估。 [35] 的探索性噪声 DQN (EN-DQN) 在动作选择期间添加噪声，并使用循环神经网络捕获影响决策的多步骤变化。 [36] 的竞争网络方法平衡在线网络和目标网络中的 Q 值更新，以减轻奖励偏差。 [37]的动态融合DDQN将先验知识纳入网络训练并调整权重以减少高估。虽然这些增强功能在一定程度上提高了 DQN 性能，但收敛缓慢和训练无效的问题仍然存在。 [38]提出的一种方法仅使用相机图像，使用具有离散动作的双深度 Q 网络（DDQN）算法来导航环境并成功避开障碍物。尽管这种方法在某些情况下效果很好，但它对离散动作的使用会影响密集或复杂环境中的性能，并且其奖励函数避免了局部最小值的挑战。同样，参考文献。 [39]使用具有离散动作的 DDQN 算法进行导航，根据距离奖励目标实现和避免碰撞。然而，这种方法需要网格地图占用作为输入，增加了运行时计算开销。相比之下，我们提出的方法优先考虑结构化奖励函数的设计，该函数包含特定领域的目标，包括目标实现、路径平滑和避障。这是通过采用综合策略来实现的，该策略将决斗架构和优先体验重播与成型奖励相结合。

PMR-Dueling DQN 方法直接解决了样本效率限制。然后，优先经验重播会根据经验的重要性对其进行优先级排序，从而确保在学习过程中有效利用罕见且重要的经验。该组合还解决了适应性的限制。决斗 DQN 允许智能体通过独立地关注学习状态和动作值来快速调整其决策过程以适应环境的变化。我们利用神经网络代替 Q 表和经验回放来改进学习，同时减少数据相关性。广泛的模拟表明该技术在高效路径规划方面是有效的。 PMR-Dueling DQN 似乎可以有效解决机器人路径规划问题。该算法在不同场景下展示了强大的性能。与幼稚的信号相比，我们特意设计了一个符合规划目标的成型奖励系统。通过将平滑、障碍和进展丰富的元素纳入最佳奖励中，指导政策克服起点计划中持续较差的局部收敛。基本上，精心塑造的强化信号是快速学习安全、有效导航方法的关键驱动力。我们的方法通过开发专门为导航动态任务设计的奖励来解决限制机器人规划深度强化学习的统计低效问题。其塑造的奖励优先考虑避障和导航，可以提供改进的实时决策能力并减轻高估问题。

3. 理论基础

在本节中，我们概述了 Dueling Deep Q-Network 和 Prioritized Experience Replay 背后的基本理论，其中我们提出的方法建立在这些理论的基础上，以提高传统 DQN 的性能。

强化学习涉及智能体与环境交互，如图 1 所示。它由智能体观察环境状态、采取行动、接收奖励和更新策略的循环过程组成。一种常见的强化学习技术是 Q 学习，它迭代地改进指导代理动作选择策略的质量值 (q) 函数。此 Q 学习更新过程遵循等式 (1) 中所示的特定数学公式，用于根据代理与环境交互期间观察到的状态、动作、奖励和下一个状态来细化 q 值。

$$q(s_t, a_t) \leftarrow q(s_t, a_t) + \eta(r_t + \gamma \max_{a'} q^*(s_{t+1}, a') - q(s_t, a_t)) \tag{1}$$

其中s_t表示当前状态，a_t表示采取的行动，η是缩放系数，r_t是观察到的奖励，s_{t+1}是达到的下一个状态，a'是下一步可能的行动，γ是未来奖励的折扣因子。然而，最优的 q^* 值最初是未知的，因此该算法依赖于当前的 q_t 估计值，通过称为引导的过程递归地替换最优的 q^*。本质上，q_t 使用自己的值在每个时间步将自身更新为改进的 q_{t+1}，逐渐收敛到更接近真正的最优 q。

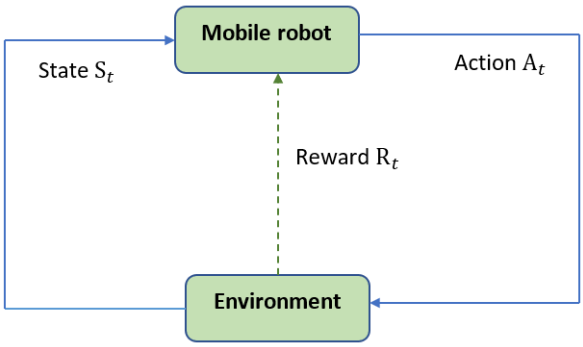


图 1. 强化学习过程。

3.1. Deep Q-Network (DQN)

DQN 通过利用深度神经网络来解决具有高维状态的复杂问题，从而改进强化学习。深度神经网络的主要工作是估计动作价值函数，从而计算每个状态下所有可用动作的 Q 值。近似函数指定为 Q(S,a;)，考虑当前状态（表示为 S）和来自动作空间的动作（表示为 a）。由 w 表示的网络权重对于确定近似的准确性至关重要。 DQN算法可以利用深度神经网络找到给定状态下Q值最高的动作。这个过程保证了智能体根据各种动作的估计值做出准确的决策。迭代训练更新网络的权重 w，从而不断改进 Q 值近似。因此，智能体通过选择与当前状态中最高 Q 值相关的动作来训练最大化其行为。

一般来说，DQN 方法应用深度学习来解决涉及复杂且连续的状态和动作空间的强化学习问题。它构建了一个价值网络和一个损失函数，损失函数测量网络预期值和期望值之间的差异[40]。目标是通过调整网络参数来最小化这种差异。训练 Q 网络需要使用标记样本。方程（2）代表了Q学习算法的修改，它是DQN的核心。

$$Q(S_t, A_t) \leftarrow a(S_t, A_t) + \alpha[R_{t+1} + \gamma \max Q(S_{t+1}, A_t) \\ -Q(S_t, A_t)] \tag{2}$$

那么，损失函数可以定义如下式（3）。

$$L(\omega) = E\left[(R_{t+1} + \gamma \max Q(S_{t+1}, A_t) - Q(S_t, A_t,\ \omega))^2\right] \tag{3}$$

在整个网络训练过程中，训练数据不断收集并作为训练标签存储在经验丰富的池中。然后使用小批量和随机梯度下降方法对经验池中的这些数据进行训练。目标是优化网络权重并求解 Q 值函数。

3.2. Deep Dueling Neural Network (Dueling DQN)

研究表明，深度 Q 学习可能会与高估的值作斗争，从而阻碍具有扩展动作和状态空间的问题的收敛速度 [14]。因此，我们采用了 Google DeepMind 在 2016 年推出的深度决斗算法 [14]，结合成形奖励函数和优先体验回放来进一步优化该系统的收敛。使深度决斗变得优越的关键创新是其神经网络架构。对于许多州来说，当这些行动的选择不会影响接下来发生的事情时，就没有必要估计行动价值[29]。深度决斗架构通过单独学习状态值和动作优势而不是直接的动作值来解决这个问题。这可以避免在无关紧要的动作上浪费计算，从而加速训练。我们使用深度决斗的动机是它有可能减轻高估并基于这种改进的神经网络结构更快地收敛到最佳 Q 值。深度决斗算法不是直接估计动作值 (Q) 函数，而是将神经网络分成两个独立的全连接层流。一个流估计状态值 $V(s；\beta)$，而另一个估计行动优势$A(s、a；\alpha)$、α 和 β 是两个流的参数。这些在输出层组合以产生最终的 Q 值，如公式 (4) 所示。

$$Q(s,\ a\ ;\ \alpha,\ \beta)\ =\ V(s;\ \beta)\ +\ A(s,\ a;\ \alpha) \tag{4}$$

通过分离价值和优势估计，深度决斗避免了对不影响后续状态的动作值的浪费计算。然而，单独的状态值 V 和动作优势 A 是不可识别的，因为网络直接输出 Q 值。为了解决这个问题，优势函数 A 通过减去所有动作 a' 的平均值来集中。这样在保证性能并提高优化稳定性的同时集中了优势。这里，$meanA(s,\ a'；\alpha)$ 表示相对于参数 α 的所有操作 a' 的平均优势。根据中心对齐优势计算出的修改后的 Q 值如公式 (5) 所示。通过集中优势，深度决斗算法可以在缺乏可识别的单独值和优势项的情况下实现稳定的优化。

$$Q(s,\ a\ ;\ \alpha,\ \beta) = V(s;\ \beta) + (A(s,\ a\ ;\ \alpha) - meanA(s,\ a'\ ;\ \alpha)) \tag{5}$$

3.3. Prioritized Experience Replay

代理的经验（包括成功和失败的尝试）在与环境交互期间被记录，并存储在经验回放单元中以用于训练目的。这些事件在整个学习过程中发生时都会记录在播放单元中。重播这些事件使智能体能够从好的和坏的行为中学习，从而不断改进其行为。利用记录的经验，代理可以了解各种操作的效果，并可以做出调整以随着时间的推移提高性能。优先体验重播根据分配的优先级从重播缓冲区中选择事件，而不是对事件进行采样

随机的。根据这些偏好，它会优先考虑体验并对其进行概率采样。通过强调更重要的事件，与经验重放中的随机采样相比，优先经验重放有可能提高学习效率。

然而，各种经验样本的意义各不相同。鉴于体验回放单元不断更新新样本，使用均匀随机采样选择少量样本作为模型的输入可能会导致利用率不足，甚至覆盖重要数据。结果，模型的训练效果受到影响。为了提高模型训练的成功率，这项工作使用了优先经验回放技术[41]。通过这种策略，优先从经验回放单元中选择样本，从而提高了接收更有意义的样本的可能性。这提高了训练过程的有效性，确保有意义的经验样本对模型的学习和整体性能做出更显着的贡献。

在学习过程中，会重新计算每个重放事件的 TD 误差，并根据新的 TD 误差来调整体验的优先级。如式（6）所示，它使用比例优先级方法来决定经验的重要性。小正整数 ϵ 确保 TD 误差接近 0 的样本被选择的机会可以忽略不计。

$$P_l = |\delta_1| + \epsilon \tag{6}$$

因此，可以使用方程（7）来计算选择经验l的可能性。在此等式中，m 表示体验回放单元的大小。参数 α 位于 [0, 1] 范围内，控制优先级利用程度。 $\alpha = 0$，采用均匀采样，而 $\alpha = 1$对应于贪婪策略采样方法。

$$P_l = \frac{p_l^\alpha}{\sum_m p_m^\alpha} \tag{7}$$

4. 方法论

在本节中，详细描述了所提出的具有成形奖励和优先体验重放的决斗深度 Q 网络（PMR-决斗 DQN）的框架以及相应奖励函数的设计。图 2 所示的移动机器人路径规划系统是增强型决斗 DQN (PMR-Dueling DQN) 系统，它优先考虑经验回放。为了取代传统的 Q 表，我们在解决方案中使用了决斗网络。为了在选择执行哪些动作时为代理提供更多信息，网络学习状态动作价值函数的两种不同表示：状态值和优势。相比之下，行动优势函数与行动相关，表示与国家有关的行动的平均感知优点。它有效地解决了奖励偏差问题。通过这种竞争性网络结构，智能体可以掌握反映环境状态的更真实的值 V(s)，而不受任何与行为相关的影响[14]。虽然 Dueling DQN 提供了架构修改来改进状态值估计，但它并没有完全解决复杂任务中奖励稀疏的潜在挑战。在本地信息受限的情况下，稀疏和延迟奖励的存在给机器人导航领域带来了巨大的挑战。例如，仅在达到目标时认可代理或惩罚与障碍物碰撞的奖励结构可能会导致各种问题。在某些情况下，训练有素的策略可能会表现出朝向目标的运动和相反方向的动作以绕过障碍物，从而有效地将智能体困在环境中的局部最小值处。此外，获得有效的逃生策略可能需要接触不同的场景。

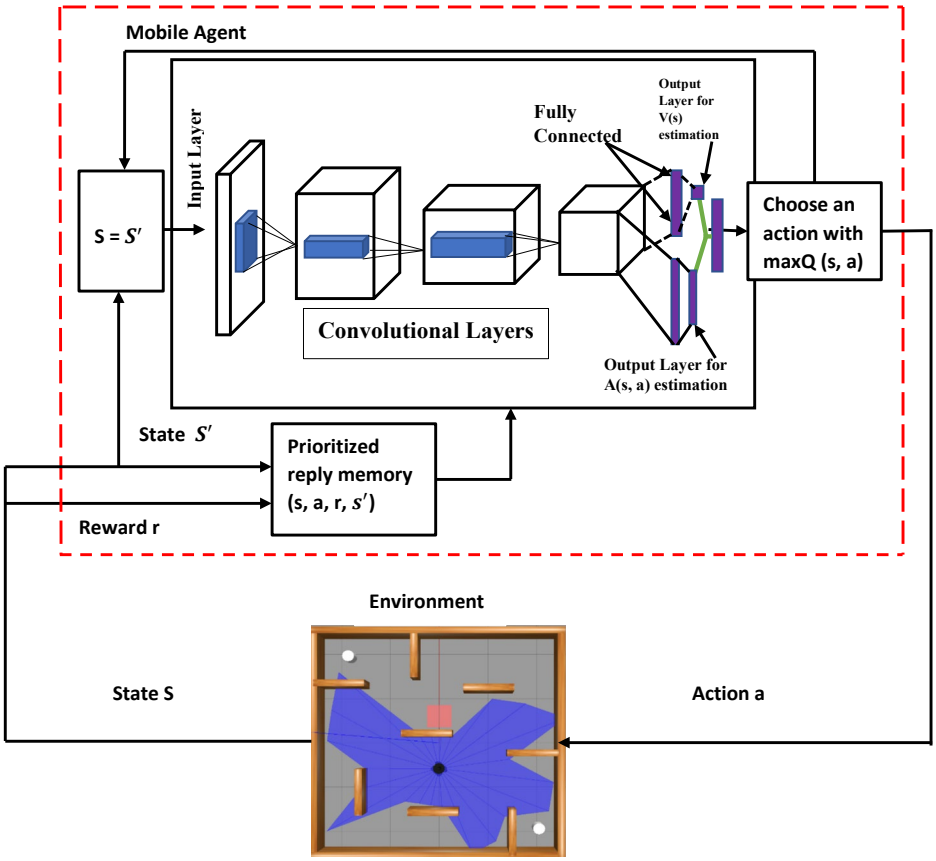


图 2. 所提出算法的框架。

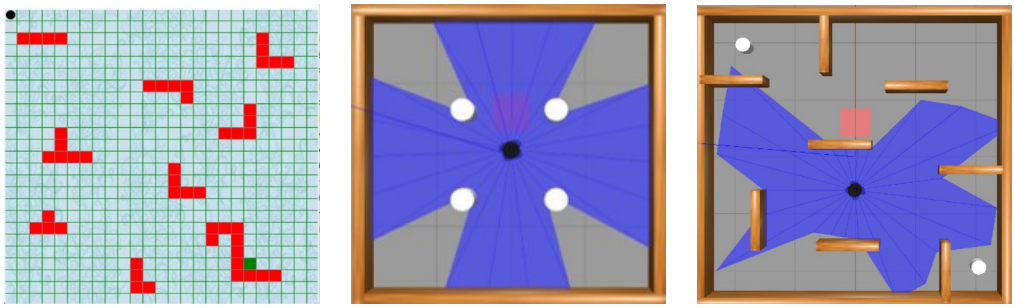
为了解决这个问题，第 4.4 节引入了成形奖励函数 *r*，通过提供更加结构化和指导性的学习过程来减轻 Dueling DQN 改进固有的限制。具体来说，不是只在完成整个任务时给予大额奖励，而是引入中间奖励来加强渐进的进度。这鼓励代理采取中间策略，最大限度地减少陷入局部最小值的风险。通过结合这种形状的奖励函数，模型会收到有针对性的反馈，影响学习动态，以强调网络可能忽视的关键方面。 Q(S, a)表示使用的近似函数，其中S表示当前状态，a表示动作空间内的动作。因此，在给定状态下，具有最高可能 Q 值的动作被选择为机器人的最佳选择。

4.1. Simulation Environments and Setup

为了验证所提出算法的有效性，我们在三个训练环境中进行了对比实验，即虚拟的25×25网格障碍物环境和两个分别名为E-1、E-2和E-3的Gazebo模拟环境。类似于现实世界环境的两个 Gazebo 模拟环境是：（1）具有静态障碍物的环境和（2）同时包含静态和动态障碍物的环境。我们使用 Gazebo 的 Turtlebot3 机器人作为在模拟环境中进行测试的平台。

使用名为 Tkinter 的 Python 内置标准库构建 GUI 环境模型 E-1。本实验有一个 25 x 25 网格障碍物环境。图3a外部的白线代表边界，而整个地图内的绿线代表网格。内部灰色表面描绘了地图上没有障碍物的区域，而红色表面则象征着困难。障碍物环境中的起始位置为左上角(1,1)，目标点为绿色，

朝向右下角 (19,21)。开发的障碍物环境由 51 个相同大小的静态障碍物组成，根据 Tkinter 工具包通过两个对角点的坐标在地图上定位。代理由代理位置内部绘制的黑色圆圈表示，代码中的地图环境可以通过修改网格坐标来定制。



(一) (二) (三)

图 3. 模拟环境：(a) 25 × 25 网格地图； (b) 有静态障碍物的凉亭环境； (c) 包含静态和动态障碍物的凉亭环境。

如图 3b、c 所示，环境 E-2 具有四个固定的圆柱形障碍物，而环境 E-3 则呈现出更复杂的设置，棕色物体充当静态障碍物，两个白色圆柱体位于环境的角落，充当移动障碍物。在 E-2 和 E-3 中，配备激光距离传感器的 Turtlebot3 作为机器人平台运行。红色矩形对象表示目标。在整个训练阶段，机器人的初始位姿被设置为训练环境地板的几何中心。目标的放置随机发生在环境中没有障碍物的位置。该实验设置有助于研究机器人在不同环境中导航、处理静态和动态障碍物的能力。 Turtlebot3平台的选择和激光传感器的具体范围有助于在不同场景下对所提出的算法进行综合评估，从而增强算法路径规划能力的鲁棒性和有效性。模拟实验的总集数在网格地图环境E-1中设置为3000，在两个Gazebo模拟环境E-2和E-3中设置为1000。当智能体达到目标或与障碍物碰撞后，情节就会终止。当代理的平均奖励达到令人满意且一致的水平时，训练结束。这项工作中使用的模型是使用表 1 中列出的软件和硬件规格进行训练的。该算法在 Python 3.7 中实现，使用 TensorFlow 作为 Keras 的后端来定义神经网络架构。利用 TensorFlow，模型可以在训练期间利用 GPU 加速。

表 1. 使用的软件和硬件配置。

Software	Hardware
Python 3.7	16 GB RAM memory
TensorFlow (backend for Keras) https://www.tensorflow.org/	Nvidia GeForce GTX 1050 GPU
Robot Operating System (ros melodic)	Intel Core i5-9300H CPU
Gazebo 9	-

4.2. State of the Environment

在本研究使用的模拟环境中，状态设计旨在总结对于有效导航和避障至关重要的关键信息。

基于 Tkinter 的网格环境 (E-1) 中的状态表示包括代理在网格内的确切位置、到目标的欧几里德距离和指标

表示障碍物与代理位置的接近程度。因此，环境 E-1 在时间步 t 的状态向量 s_t 可以表示为：

$$S_t = (x_{agent}, y_{agent}, d_{target}, O_{proximity}) \tag{8}$$

其中 x_{agent} 和 y_{agent} 表示代理在网格内位置的笛卡尔坐标。 $O_{proximity}$ 是一个二进制指示向量，表示代理附近是否存在障碍物。 d_{target}——代理和目标点之间的欧几里德距离——将是：

$$d_{\text{target}} = \sqrt{(x_{\text{agent}} - x_{\text{target}})^2 + (y_{\text{agent}} - y_{\text{target}})^2} \tag{9}$$

其中x_{agent}、y_{agent}表示代理的笛卡尔坐标，x_{target}、y_{target}表示目标的笛卡尔坐标。

相比之下，Gazebo 模拟环境（E-2 和 E-3）包含更动态的状态表示，其中包含 Turtlebot3 机器人的姿态、来自激光测距传感器的有关不同方向障碍物距离的信息、静止和移动障碍物的坐标和属性以及目标位置与障碍物的距离。 Gazebo 模拟中正确配置了七个激光雷达点。给定值在控制系统所需的输入和计算资源的使用之间取得平衡。通过仔细选择激光雷达点，控制系统可以在保持计算效率的同时获得相关的环境数据。通过对周围环境的全面了解，控制系统可以利用状态信息来评估环境，识别任何障碍物，并绘制出周围的最佳路线。激光雷达传感器精确而全面的感知能力显着增强了机器人的整体性能，使其能够安全有效地在充满挑战的环境中航行。这些状态变量经过精心选择，为强化学习代理提供相关的环境信号，以便其能够在模拟环境中做出决策并避开障碍物。激光扫描仪覆盖从 0 到 360 度的角度 θ 的检测范围，从而创建如图 4 所示的 360 维数据表示。为了解决处理高维的挑战，同时确保捕获环境细节，我们将检测距离限制在 0.12 m 到 3.5 m 的范围内。

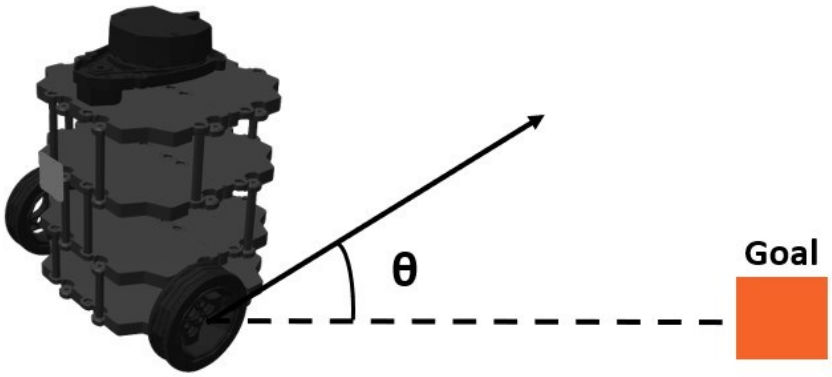


图 4. 具有当前位置和目标航向的机器人方向：基于传感器的导航。

$$\text{pose}_{robot} = (x, y, \theta) \tag{10}$$

其中 x 是 Turtlebot3 机器人的 x 坐标； y 是 Turtlebot3 机器人的 y 坐标； θ 是机器人当前位置和到目标方向之间的角度。因此，对于 Turtlebot3 配备激光距离传感器的环境（E-2 和 E-3），状态 s_t 可以表示如下。

$$S_t = (\text{pose}_{\text{robot}} + \text{sensor}_{\text{laser}}(7points) + \text{obstacle}_{\text{info}} + d_{\text{target}}) \tag{11}$$

其中$sensor_{laser}$包含来自激光测距传感器的数据，提供从七个激光雷达点到周围障碍物的多个方向的距离测量。 $barrier_{info}$ 包含有关静态和移动障碍物的详细信息，例如它们的位置、形状和属性。 d_{target}表示机器人当前位置到指定目标点的距离。

4.3. Network Structure

使用 Tensorflow 库是因为它的开源性质和提供多功能机器学习工具的能力。其 API 能够构建具有可调整架构的模型和应用程序。 Keras 用于对网络层进行编程，减少了构建控制系统所需的代码量。

图5所示的网络模型架构是按步骤规划的。该网络由一个输入层组成，该输入层实现密集 ReLU 激活以接受状态表示输入数据。这会输入两个卷积层，从输入状态中提取空间特征。然后，输出被展平为一维向量，并分成单独的优势流和价值流。优势流和价值流都包含具有 ReLU 激活的密集层，以分别估计优势函数和价值函数。最终的优势函数和价值函数由聚合层组合后，然后传递到线性激活输出层，表示给定输入状态下每个可能动作的 Q 值。使用线性输出激活允许网络预测不受任何范围限制的 Q 值。

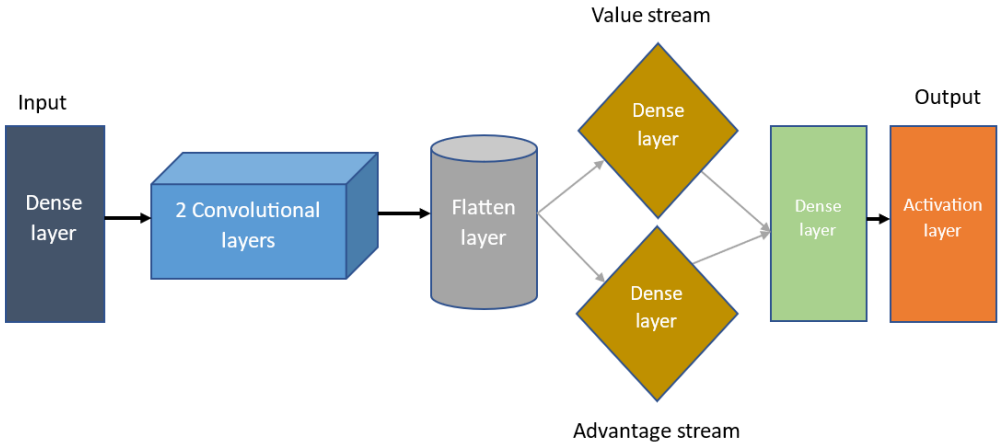


图 5.PMR-Dueling DQN 网络的架构。

网络模型具有一个输入层，其中包含关键状态信息以通知输出层的操作选择。根据代理是在 Gazebo 模拟环境还是在抽象网格世界地图中运行，这些输入和输出层的尺寸略有不同。

在 Gazebo 3D 模拟器中，输入到输入层的状态表示由 10 个变量组成，这些变量描述了代理的当前位置、方向、最近速度、目标位置和障碍物接近度等方面。因此，输入层包含 64 个神经元来匹配这个 10 维状态空间表示。

Gazebo 环境的输出层由 7 个神经元组成，每个神经元编码一种允许的操作。线速度控制有 2 种动作（前进/后退），角速度控制有 5 种动作（左转/右转和三种不同的转弯速度）。当智能体交替放置在 2D 网格世界地图中时，状态表示仅包含 4 个维度——智能体和目标的 x-y 坐标。相应地，输入层有 64 个神经元来匹配这个简化的 4 维状态空间。网格世界只提供 4 种移动动作——上、下、左、右。因此，输出层由 4 个神经元组成，每个神经元对这些离散导航方向之一进行编码。

4.4. Design of Shaped Reward Function

在强化学习环境中，智能体的行为很大程度上受到奖励函数的影响。所提出的奖励函数总结了专为移动机器人导航任务量身定制的对齐、距离和避障奖励的独特平衡。与通常关注单一方面的单一奖励结构不同，我们的方法合并了这些组成部分，以整体解决路径规划中的挑战。这种独特的组合旨在激励智能体的行为，不仅鼓励朝着目标前进，而且确保最佳的方向对准、有效的路径选择和熟练的避障，从而为移动机器人提供更全面的学习框架。在本文中，我们提出了网格地图环境（E-1）的奖励函数，名为r_{env_1}，以及Gazebo环境（E-2和E-3），名为r_{env}

$_{2,3}$， 分别。

每个时间步的小奖励、撞上障碍物的惩罚或完成多个步骤以达到目标而不发生碰撞的稀疏奖励可能足以训练代理尝试探索其环境，同时避免手头任务的障碍。因此，对于环境E-1，我们计算3000个episode的奖励（成本），奖励函数定义为等式（12）：

$$r_{\mathrm{env}_1} = \begin{cases} 10, & \text{if the agent reaches the target} \\ 0, & \text{if the agent reaches any position} \\ -1, & \text{if the agent reaches an obstacle} \end{cases} \tag{12}$$

$r_{env_{2,3}}$ 的奖励函数由三个部分组成；对齐奖励r_a、距离奖励r_d和避障奖励r_{ob}。我们提出了与机器人的操作特征相一致的奖励函数，同时考虑到与训练 Turtlebot3 的模拟中使用的机器人相关的特定值。这些值包括 0.2 m/s 或 0.4 m/s 的线速度以及 $\pm\frac{\pi}{6}$ rad/s、$\pm\frac{\pi}{12}$ rad/s、0 rad /s 的角速度。机器人向目标的前进直接受到其线速度的影响。我们使距离奖励 (r_d) 组件对线速度变化更加敏感，以解决这一问题。然后，我们扩展了对齐奖励（r_a）组件以考虑角速度方差，同时考虑到机器人可以处理的角速度范围。考虑到特定的角速度值，奖励现在受到当前航向和目标角度之间的差异的影响。按1000集计算。鼓励智能体通过 r_a 将其方向与目标方向保持一致。它是根据代理的航向和朝向目标的角度之间的差异来计算的。当角度偏离理想对齐时，奖励会下降，并且当智能体笔直前往目标时奖励最高。

$$r_a = 1 - 4\left|0.5 - \left(0.25 + 0.5 \times \frac{(\theta\%(2\pi))}{\pi}\right)\right| \tag{13}$$

其中 θ 是当前航向和目标角度之间的差值。因此，当 θ 的值除以 2π 时，$\theta\%$ (2π) 将产生余数。结果将是 0 到 2π 之间的值。

前任 ponential函数用于计算距离rewa rd r_d。

$$r_d = 2^{\left(\frac{d_c}{d_g}\right)} \tag{14}$$

其中 d_c 是代理与目标之间的当前距离，d_g 是代理与目标之间的期望或最佳距离。

避障惩罚 r_{ob} 是对过于接近障碍物的设定负奖励：

$$r_{ob} = \begin{cases} -5, & \text{if } min_range < 0.5 \\ 0, & otherwise \end{cases} \tag{15}$$

其中 min_range 是代理到其传感器检测到的最近障碍物的最小距离。它是根据环境中激光测距仪的数据计算得出的。结合方程（13）-（15），总奖励函数可以用方程（16）计算如下。

$$r = \left(round(r_a[\text{action}] \times 5) \times 2^{\left(\frac{d_c}{d_g}\right)} \right) + r_{ob} \tag{16}$$

其中应用 $round$ 函数来确保计算的奖励表示为离散值，从而增强奖励信号的可解释性。

因此，Gazebo 模拟环境中代理的整个奖励函数如方程（17）所示。

$$r_{env_{2,3}} = \begin{cases} 500, & d_c = d_g \\ r + r_{ob}, & d_c < d_g < d_{ob} \\ -100, & d_o b < d_g \end{cases} \tag{17}$$

其中 d_{ob} 是当前距离与障碍物之间的距离。

因此，网格地图和 Gazebo 模拟环境的总奖励函数；r_{env_1}和$r_{env_{2,3}}$分别在等式(18)中定义如下。

$$r_t = \begin{cases} r_{\text{env}_1} = \begin{cases} 10, & \text{agent reaches the target} \\ 0, & \text{agent reaches any position} \\ -1, & \text{agent reaches an obstacle} \end{cases} \\ r_{env_{2,3}} = \begin{cases} 500, & d_c = d_g \\ r + r_{ob}, & d_c < d_g < d_{ob} \\ -100, & d_o b < d_g \end{cases} \end{cases} \tag{18}$$

5. 绩效评估

在本节中，介绍了所提出的 PMR-Dueling DQN 方法的参数设置、仿真结果和性能比较分析，如算法 1 所示。该方法与经典的 Q-Learning、DQN 和 DDQN 算法进行了比较。

5.1. Parameter Settings

经过多次实验，表 2 中建议了最佳参数，这些参数是学习的适度值，表明该算法确实会保留旧信息，同时仍然能够获取新数据。α 和 β 分别是控制优先级和重要性采样量的参数。

表 2. 两种环境类型中的超参数及其值。

Parameter	For E-1	For E-2 and E-3
Number of episodes	3000	1000
Learning rate	0.01	0.00025
Discount factor	0.9	0.99
Minibatch size	32	128
Replay memory size	10,000	10,000
α	0.5	0.6
β	0.5	0.4

Algorithm 1 Dueling Deep Q-Network with shaped Rewards and Prioritized Experience Replay (PMR-Dueling DQN).

Input: minibatch m, replay period K, action advantage parameter α, state value parameter β, budget T
Initialize replay memory D, weight-change Δ
Initialize Dueling Q-Network parameters θ
Initialize the target Dueling Q-Network parameters $\hat{\theta} = \theta$
Initialize r_t of Equation (18)
Observe initial state S_0 and select initial action A_0 using the online network $\pi_0(A_0|S_0)$
for $t = 1$ to T **do**
    Observe state s_t, reward r_t, next state s_{t+1}, and random action a_t
    Store transition (s_t, a_t, r_t, s_{t+1}) in the replay memory D with an initial priority value.
    Combine the advantage and value functions as follows:

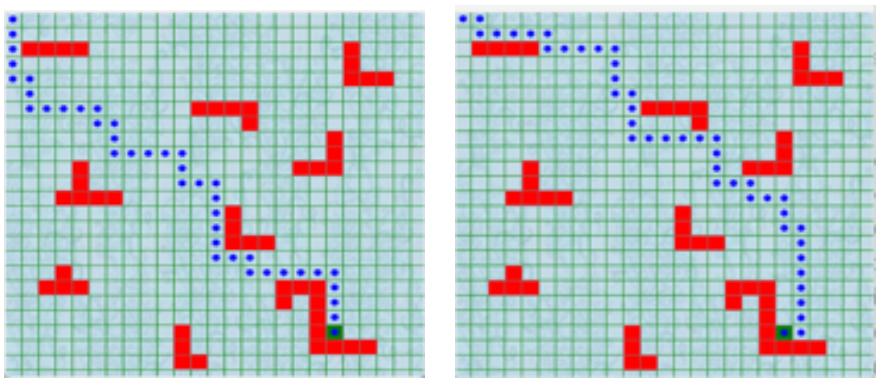
$$Q(s, a; \alpha, \beta) = V(s; \beta) + (A(s, a; \alpha) - \text{mean}A(s, a', \alpha))$$

    if t mod $K = 0$ **then**
        for $j = 1$ to m **do**
            Sample a transition (s_j, a_j, r_j, s_{j+1}) based on the priorities $P(j) \propto \frac{P_j^{\alpha}}{\sum_j P_j^{\alpha}}$
            Update the priority of the transition to $|\delta_j|$
        end for
    end if
    Update the network $\hat{\theta} = \theta$
end for

5.2. Simulation Results and Performance Analysis

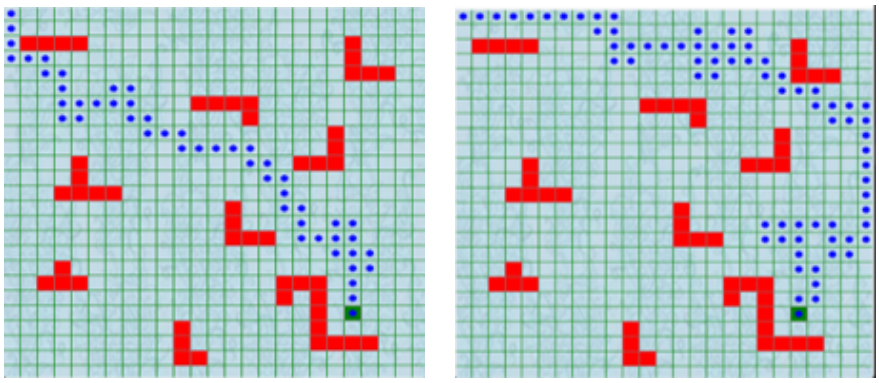
5.2.1。对于网格地图环境 (E-1)

移动代理负责避免与障碍物发生碰撞。在整个过程中，代理可以执行四种操作之一：向上移动、向下移动、向右移动或向左移动。边界和角度位置是例外，因为一个或多个操作选项不可用。此外，代理只能在单个操作中在每条选定的路线中采取一步。所提出的环境被划分为多个单元，其中一些空间被障碍物填充。如果移动代理撞到其中一个，则被视为碰撞（负奖励）。图 6 显示了模型分别针对所提出的 PMR-Dueling DQN 算法、DDQN 算法、DQN 算法和 Q 学习算法进行训练后获得的路径。如图所示，PMR-Dueling DQN 算法的路径规划结果优于其他三种算法，而 DDQN 的性能优于 DQN。 DQN 的性能也优于经典的 Q 学习算法。



(**a**) (**b**)

图 6.*Cont*。



(**c**) (**d**)

图 6. 智能体绘制的最短路线：(a) 对于 PMR-Dueling DQN 算法； (b) 对于DDQN算法； (c) 对于DQN算法； (d) Q 学习算法。

表 3 显示了最终的 Q 表以及 PMR-Dueling DQN 算法的最终路线中各自的最佳操作。每次运行中的训练过程都会生成一个 Q 表，它代表每个位置所需的操作。表中所示的四个动作按照以下顺序：向上动作、向下动作、向右动作、向左动作。完整表格的 Q 值表示智能体在观察环境后决定的最佳操作。智能体完成完整动作后实现目标的一些最终动作序列如下所示：down-down-down-down-right-down-down-right-right-right-right-down-right-down-down-right-right-right-right-down-down。。。显然，Q 表存储了强化学习代理通过探索环境所学到的关于最优策略的知识。分析该表可以深入了解如何评估状态以及在某些状态下首选采取哪些行动。当智能体探索状态并通过更新规则细化 Q 值时，就会学习到有用的状态到动作的映射。检查在不同学习阶段生成的表格可以更好地理解这些映射如何从大部分随机演变为更优化的重点政策。学习到的 Q 表本质上代表了智能体的总结经验，决定了其行为策略。

表 3. PMR-Dueling DQN 算法的最终 Q 表值及其各自的最佳操作。

Status	Action to Up	Action to Down	Action to Right	Action to Left
[3.0, 3.0, 17.0, 17.0]	1.130×10^{-9}	$\mathbf{4.159 \times 10^{-8}}$	1.698×10^{-10}	9.773×10^{-10}
[3.0, 23.0, 17.0, 37.0]	1.320×10^{-9}	$\mathbf{3.298 \times 10^{-7}}$	1.006×10^{-12}	2.624×10^{-9}
[23.0, 23.0, 37.0, 37.0]	1.234×10^{-10}	$\mathbf{2.894 \times 10^{-7}}$	-7.449×10^{-1}	1.956×10^{-9}
[43.0, 23.0, 57.0, 37.0]	3.917×10^{-11}	$\mathbf{5.689 \times 10^{-7}}$	-5.742×10^{-1}	9.710×10^{-10}
[63.0, 23.0, 77.0, 37.0]	3.157×10^{-11}	-6.656×10^{-1}	$\mathbf{1.277 \times 10^{-6}}$	1.868×10^{-8}
[83.0, 23.0, 97.0, 37.0]	4.259×10^{-9}	$\mathbf{6.448 \times 10^{-1}}$	-2.860×10^{-6}	1.546×10^{-8}
[103.0, 23.0, 117.0, 37.0]	8.304×10^{-10}	$\mathbf{5.073 \times 10^{-6}}$	6.161×10^{-10}	6.654×10^{-8}
[123.0, 23.0, 137.0, 37.0]	6.029×10^{-9}	4.260×10^{-8}	$\mathbf{1.325 \times 10^{-5}}$	1.388×10^{-7}
[143.0, 23.0, 157.0, 37.0]	4.530×10^{-10}	2.704×10^{-8}	$\mathbf{1.460 \times 10^{-5}}$	3.583×10^{-7}
[143.0, 43.0, 157.0, 57.0]	8.547×10^{-7}	5.570×10^{-7}	$\mathbf{4.352 \times 10^{-5}}$	3.976×10^{-8}
[143.0, 63.0, 157.0, 77.0]	2.156×10^{-6}	9.055×10^{-7}	$\mathbf{1.116 \times 10^{-4}}$	5.704×10^{-8}
[163.0, 63.0, 177.0, 77.0]	1.889×10^{-7}	$\mathbf{2.250 \times 10^{-4}}$	2.762×10^{-6}	4.763×10^{-6}
[163.0, 83.0, 177.0, 97.0]	6.753×10^{-6}	4.379×10^{-8}	$\mathbf{7.159 \times 10^{-4}}$	8.159×10^{-6}
[163.0, 103.0, 177.0, 117.0]	4.346×10^{-6}	$\mathbf{8.272 \times 10^{-4}}$	4.272×10^{-5}	7.689×10^{-7}
[163.0, 123.0, 177.0, 137.0]	3.355×10^{-5}	$\mathbf{1.447 \times 10^{-3}}$	1.354×10^{-7}	2.073×10^{-6}
[163.0, 143.0, 177.0, 157.0]	7.454×10^{-5}	2.501×10^{-8}	$\mathbf{7.130 \times 10^{-5}}$	7.825×10^{-6}
[163.0, 163.0, 177.0, 177.0]	9.339×10^{-5}	4.254×10^{-5}	$\mathbf{4.254 \times 10^{-5}}$	1.258×10^{-6}
[163.0, 183.0, 177.0, 197.0]	2.264×10^{-4}	1.271×10^{-5}	$\mathbf{7.072 \times 10^{-3}}$	5.077×10^{-6}

表 3.*Cont*。

Status	Action to Up	Action to Down	Action to Right	Action to Left
[183.0, 183.0, 197.0, 197.0]	4.483×10^{-5}	1.283×10^{-5}	$\mathbf{1.138 \times 10^{-2}}$	3.193×10^{-4}
[203.0, 183.0, 217.0, 197.0]	7.260×10^{-5}	5.061×10^{-5}	$\mathbf{1.798 \times 10^{-2}}$	7.168×10^{-4}
[223.0, 183.0, 237.0, 197.0]	7.499×10^{-5}	$\mathbf{2.680 \times 10^{-2}}$	2.042×10^{-4}	7.128×10^{-4}
[223.0, 203.0, 237.0, 217.0]	1.380×10^{-3}	$\mathbf{4.175 \times 10^{-2}}$	3.859×10^{-5}	1.057×10^{-4}
[243.0, 203.0, 257.0, 217.0]	2.553×10^{-4}	2.806×10^{-4}	$\mathbf{5.385 \times 10^{-2}}$	2.373×10^{-3}
[263.0, 203.0, 277.0, 217.0]	1.580×10^{-4}	2.287×10^{-4}	$\mathbf{7.279 \times 10^{-2}}$	4.591×10^{-3}
[283.0, 203.0, 297.0, 217.0]	7.453×10^{-5}	$\mathbf{2.299 \times 10^{-2}}$	9.690×10^{-3}	3.823×10^{-3}
[303.0, 203.0, 317.0, 217.0]	1.127×10^{-3}	$\mathbf{1.251 \times 10^{-1}}$	3.131×10^{-3}	5.609×10^{-3}
[303.0, 223.0, 317.0, 237.0]	1.384×10^{-2}	$\mathbf{3.561 \times 10^{-1}}$	1.566×10^{-3}	2.770×10^{-3}
[323.0, 223.0, 337.0, 237.0]	3.099×10^{-3}	$\mathbf{1.901 \times 10^{-1}}$	3.058×10^{-3}	1.593×10^{-2}
[323.0, 243.0, 337.0, 257.0]	2.888×10^{-2}	$\mathbf{1.002 \times 10^{-1}}$	2.254×10^{-2}	4.244×10^{-3}
[343.0, 243.0, 357.0, 257.0]	6.546×10^{-3}	2.624×10^{-2}	$\mathbf{1.565 \times 10^{-1}}$	3.689×10^{-2}
[343.0, 263.0, 357.0, 277.0]	3.308×10^{-2}	2.820×10^{-3}	$\mathbf{3.011 \times 10^{-1}}$	7.740×10^{-3}
[363.0, 263.0, 377.0, 277.0]	1.417×10^{-2}	$\mathbf{3.412 \times 10^{-1}}$	1.013×10^{-2}	5.315×10^{-2}
[363.0, 283.0, 377.0, 297.0]	6.754×10^{-2}	3.832×10^{-3}	$\mathbf{7.472 \times 10^{-1}}$	3.249×10^{-3}
[363.0, 303.0, 377.0, 317.0]	9.205×10^{-2}	4.283×10^{-2}	$\mathbf{3.097 \times 10^{-1}}$	3.811×10^{-2}
[363.0, 323.0, 377.0, 337.0]	9.063×10^{-2}	2.581×10^{-2}	$\mathbf{4.773 \times 10^{-1}}$	1.766×10^{-2}
[383.0, 323.0, 397.0, 337.0]	2.752×10^{-2}	5.310×10^{-2}	$\mathbf{3.690 \times 10^{-1}}$	1.023×10^{-2}
[383.0, 343.0, 397.0, 357.0]	1.106×10^{-1}	6.341×10^{-2}	$\mathbf{5.903 \times 10^{-1}}$	2.969×10^{-2}
[383.0, 363.0, 397.0, 377.0]	7.694×10^{-3}	$\mathbf{7.639 \times 10^{-1}}$	3.576×10^{-3}	-5.851×10^{-2}
[383.0, 383.0, 397.0, 397.0]	2.604×10^{-3}	$\mathbf{4.860 \times 10^{-1}}$	2.006×10^{-2}	-2.970×10^{-2}
[383.0, 383.0, 397.0, 397.0]	2.604×10^{-3}	$\mathbf{4.860 \times 10^{-1}}$	2.006×10^{-2}	-2.970×10^{-2}
[383.0, 403.0, 397.0, 417.0]	8.078×10^{-2}	$\mathbf{1.000 \times 10^{+00}}$	3.603×10^{-1}	-3.573×10^{-1}

粗体值表示沿整个路径选择的最佳操作。以红色突出显示的值表示最终 Q 表的初始值。绿色突出显示最终目标状态的最终值，其值为 “ +1 ”。

修改Q值的技术是四种算法之间最本质的区别。具有修改奖励和优先体验重放的决斗深度 Q 网络 (PMR-Dueling DQN) 算法通过结合深度神经网络、决斗架构、成形奖励和优先体验重放，更加重视那些提供有价值学习机会的体验，它可以直接从给定导航环境中的原始感知数据中有效地学习控制策略，从而与 DDQN、DQN 和 Q 学习相比，改进训练并可能做出更好的动作选择决策。与表格方法相比，这提供了更好的数据效率。 PMR-Dueling DQN 还通过利用决斗架构来单独学习状态值和动作优势，从而改进了原始 DQN 算法。这有助于减轻高估偏差，这是困扰传统 Q 学习和 DQN 的问题。

在训练开始时，特意减慢了代理从一个地方移动到另一个地方的可见性，以便在调整变量的同时可以查看和分析其性能。所有四种算法在相同的超参数下总共进行了 35 次实验，以获得最优结果。

如图 7 所示，PMR-Dueling DQN 算法在到达目的地方面明显优于其他方法，仅在 1000 个训练集内收敛。 DDQN 需要额外 370 集才能找到目标，而基本 DQN 需要额外 700 集。即使运行了 3000 集，表格 Q 学习也没有表现出明显的收敛趋势，凸显了其样本效率较差。 PMR-Dueling DQN 结合了深度神经网络、决斗架构、优先体验重放和形状奖励，使其能够比 DQN、DDQN 和表格 Q 学习更快、更稳定地找到最佳导航策略。

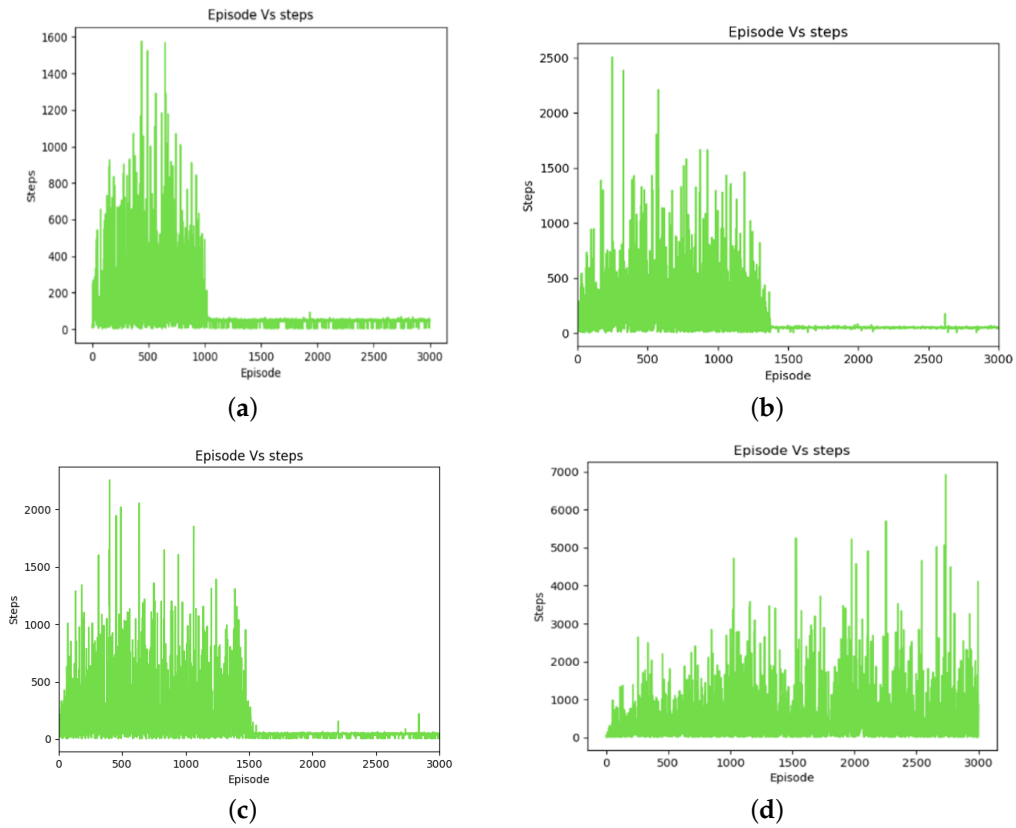


图 7. 各片段步长变化的训练图：(a) 对于 PMR-Dueling DQN 算法； (b) 对于DDQN算法； (c) 对于DQN算法； (d) Q 学习算法。

图 8 显示，PMR-Dueling DQN 算法在最大化累积奖励方面显然取得了最佳性能，在整个训练周期中达到了 10 的最大累积奖励。这种高平均奖励表明 PMR-Dueling DQN 能够快速稳定地收敛到最佳导航策略。 DDQN 获得了不错但较低的平均累积奖励 8，需要更多的时间来学习有效的策略。 DQN 的表现更差，由于其在训练过程中倾向于高估值，仅累积了最大平均奖励 6。与此同时，传统的Q-learning表现出极差的累积奖励，几乎所有训练集都获得负成本，并且完全找不到任何正奖励策略。这一比较凸显了 PMR-Dueling DQN 的显着优势，它将成形奖励和优先体验重播等创新与深度神经网络相结合，在机器人导航的样本效率和策略最优性方面大大优于 DQN、DDQN 和 Q 学习方法。

表 4 总结了网格地图模拟环境中的实验结果。在确定理想路径方面，仿真结果表明PMR-Dueling DQN优于其他三种算法。从结果中可以得出结论，带有修改奖励和优先体验重放的决斗深度 Q 网络 (PMR-Dueling DQN) 算法比经典的 Q 学习、DQN 和 DDQN 算法更适合紧急路径规划。

表 4. PMR-Dueling DQN、DDQN、DQN 和 Q-learning 之间的性能比较。

Performance Parameters	PMR-DQN	DDQN	DQN	Q-Learning
Episodes	3000	3000	3000	3000
Shortest path steps	40	43	72	103
Longest path steps	1577	1798	3579	5909
Success rate (%)	94.3	83.7	62.84	37.43

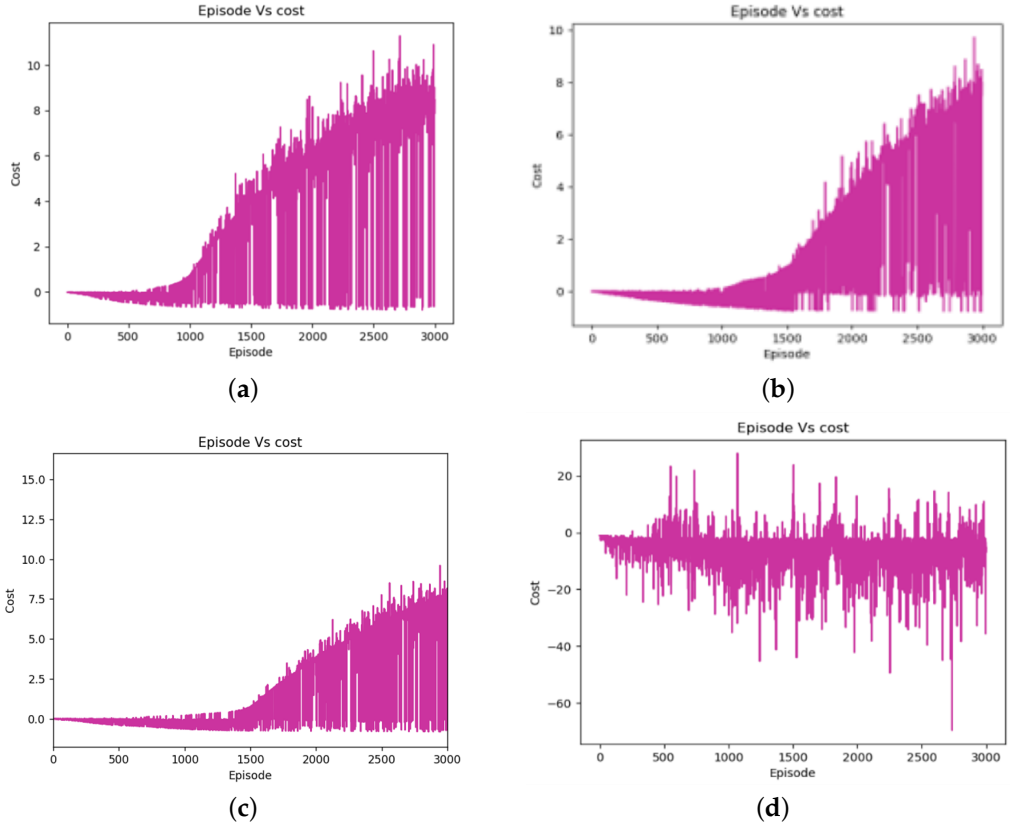


图 8. 各集成本的训练图：(a) 对于 PMR-Dueling DQN 算法； (b) 对于DDQN算法； (c) 对于DQN算法； (d) Q 学习算法。

5.2.2.对于 Gazebo 模拟环境（E-2 和 E-3）

在测试环境E-2和E-3中，分别选择了六个和七个导航点。机器人的任务是独立探索这些陌生的环境，从初始点O开始移动。在E-2的例子中，机器人的路径涵盖了位置A到F，然后返回到最终目标位置A，所有这些都在没有遇到碰撞的情况下实现。同样，对于 E-3，机器人的轨迹包括位置 A 到 G，然后返回到最终目标位置 A，同时确保无碰撞运动，如图 9 所示。局部规划器在处理静态和动态障碍物方面需要具有鲁棒性。它可以有效地利用传感器数据来检测动态障碍物并做出反应。在规划过程中，全局规划器会考虑静态障碍物（来自地图）和动态障碍物的当前状态（来自 Turtlebot3 机器人的激光雷达传感器）来生成目标序列。本地规划器持续监视环境 E-3 中动态障碍物的变化，并相应地调整其路径，这涉及如果规划轨迹上突然出现障碍物则重新规划路径。例如，在图9b中，动态障碍物阻挡了F点和G点之间的路径；本地规划器检测到该障碍物并重新规划一条新的无碰撞路径以到达下一个目标点。全局路径保持不变，但局部规划器动态调整特定运动以处理环境变化，例如移动障碍物。

为了验证所提出的 PMR-Dueling DQN 算法的效率，我们在两种 Gazebo 模拟环境中进行了对比实验。代理的每种算法的累积奖励显示在两个单独的图中，每个图对应一个环境。图 10 显示了环境 E-2 中每种算法的累积奖励，图 11 显示了环境 E-3 中每种算法的累积奖励。图 12 比较了每种环境下四种算法的平均奖励。如图所示，在不太复杂的环境E-2中，

PMR-Dueling DQN 算法再次明显优于其他算法，与 Q-learning 的 200、DQN 的 600 和 DDQN 的 1100 相比，收敛到明显更高的奖励 1400。在更复杂的 E-3 环境中，性能差距甚至更大，PMR-Dueling DQN 获得了 1200 的奖励，而 Q-learning、DQN 和 DDQN 的奖励则低得多，分别为 400、700 和 900。 PMR-Dueling DQN 在两种环境中始终如一的卓越性能证明了其更有效地学习最优策略的能力。 Q-learning 的简单性限制了其性能，而 DQN 和 DDQN 显示出改进，但仍然被提议的 PMR-Dueling DQN 利用决斗架构和优先体验重播进行样本高效学习的能力所超越。

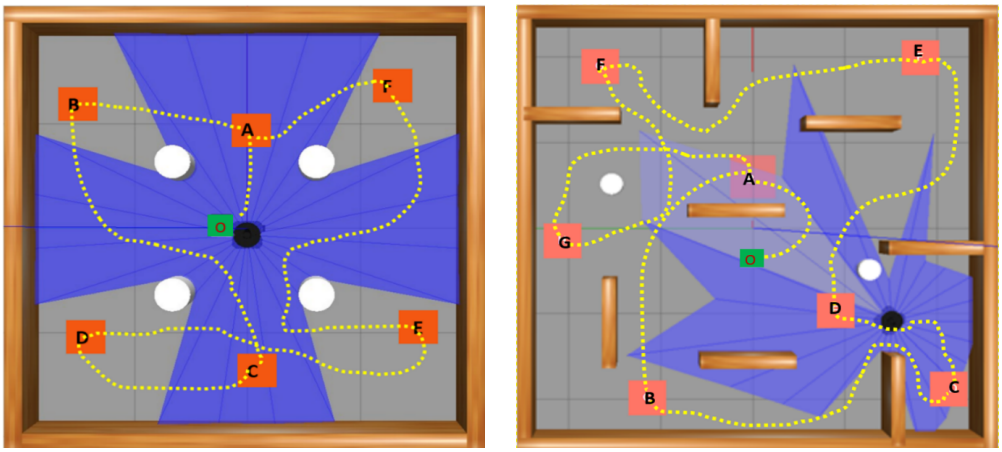


图 9. 所提出的 PMR-Dueling DQN 算法的生成路径： (a) 在仅包含静态障碍物的环境 E-2 中。机器人的轨迹由A位置到F位置组成，最终回到A位置的最终目的地； (b) 在包含静态和动态障碍物的环境 E-3 中。机器人沿着包含位置A到G的路径行驶，然后返回到最终目标位置A。

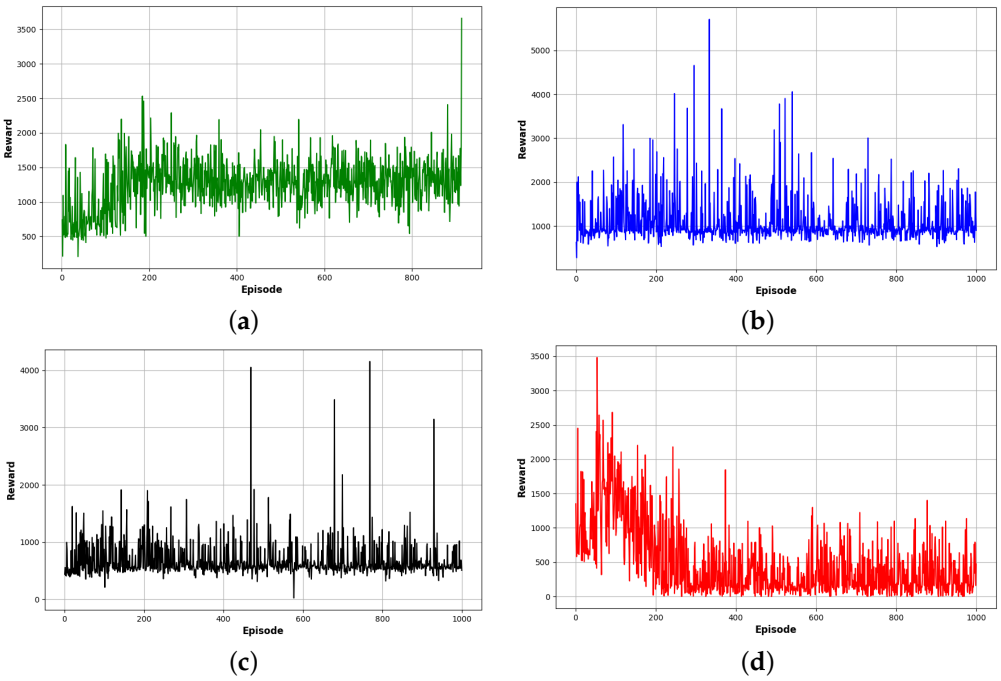


图 10. 环境 E-2 中每种算法的累积奖励：(a) PMR-Dueling DQN 算法； (b) 对于DDQN算法； (c) 对于DQN算法； (d) Q 学习算法。

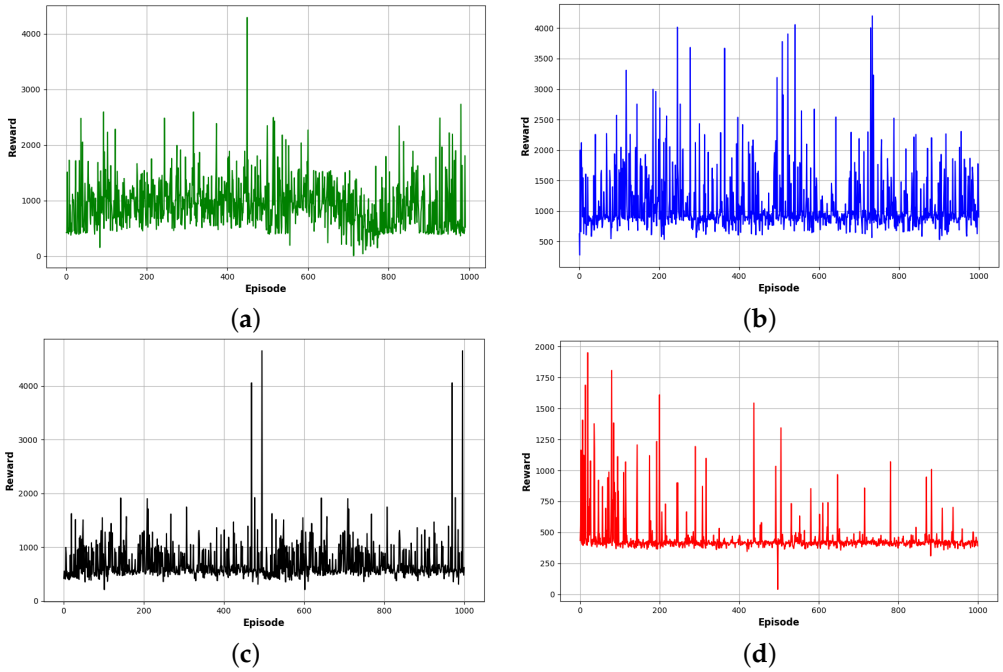


图 11. 环境 E-3 中每种算法的累积奖励：(a) PMR-Dueling DQN 算法； (b) 对于DDQN算法； (c) 对于DQN算法； (d) Q 学习算法。

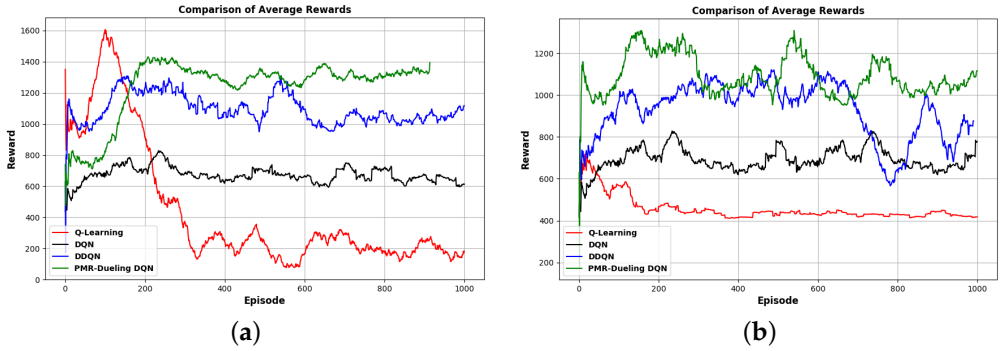


图 12. 四种算法的机器人奖励比较：(a) 在环境 E-2 中，仅包含静态障碍物；(b) 在环境 E-3 中，同时包含静态障碍物和动态障碍物。

表5比较了用于路径规划任务的Q学习算法、DQN算法、DDQN算法和PMR-Dueling DQN算法的成功率和收敛时间。表5给出了四种算法在仅包含静态障碍物的环境E-2下到达目标点的成功率和收敛时间。 PMR-Dueling DQN算法较DDQN算法成功率提升28.2%，较DQN算法提升35.3%，较Q-learning算法提升57.9%。表5还比较了这四种算法的收敛速度。结果发现，PMR-Dueling DQN 在所有方法中收敛到目标的速度最快，需要 107 分钟，而 Q-learning 需要 268 分钟，DQN 需要 139 分钟，DDQN 需要 116 分钟。总体而言，PMR-Dueling DQN 获得最终目标点所需的时间最少，这表明与其他分析的算法相比，任务学习速度更快。

表5还显示了四种强化学习算法在更复杂的环境E-3（同时包含静态和动态障碍物）下的相同定量评估比较。所有四种算法的性能都会随着动态障碍物的加入而降低。然而，它仍然表明 PMR-Dueling

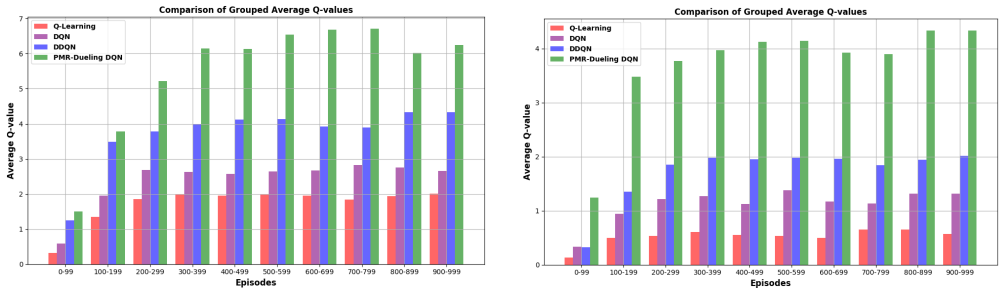
与其他三种方法相比，DQN 的成功率最高，比 DDQN 算法成功率提高了 38.1%，比 DQN 算法提高了 47.0%，比 Q-learning 算法成功率提高了 58.2%。表5还列出了四种算法在特定任务上的收敛速度。结果表明，PMR-Dueling DQN收敛速度最快，在122分钟内达到了目标。这比 Q-learning 更快，Q-learning 需要 276 分钟才能收敛；比 DQN 算法更快，需要 161 分钟；并且也比 DDQN 更快，后者花费了 143 分钟。这证明了 PMR-Dueling DQN 的卓越性能，它在动态和静态版本的环境中都达到了最快的收敛。总之，这些结果凸显了 PMR-Dueling DQN 的卓越性能，根据表中所示的收敛时序，该任务在此任务上显示出更快的收敛和学习速度。

表5.不同环境下算法的比较。

Method	Success Rate	Convergence Time/min
Algorithm comparisons on environment E-2		
Q-learning	26.7	268
DQN	49.3	139
DDQN	56.4	116
PMR-Dueling DQN	**84.6**	**107**
Algorithm comparisons on environment E-3		
Q-learning	21.4	276
DQN	32.6	161
DDQN	41.5	143
PMR-Dueling DQN	**79.6**	**122**

粗体值 指示提议的 PMR-Dueling DQN 的性能值 方法。

图 13 中所示的图表分为每个间隔 100 个片段，并且对于每个间隔，每种算法的平均 Q 值显示为分组条形。这种视觉表示可以深入了解这些算法如何随着时间的推移执行以及它们的 Q 值在训练过程中如何演变。条形图显示，与两种环境中的其他算法相比，PMR-Dueling DQN 实现了显着更高的最大平均 Q 值。在 E-2 中，它获得的最大 Q 值为 6.75，Q 学习为 2，DQN 为 2.8，DDQN 为 4.36。 E-3 中的优势更加明显，PMR-Dueling DQN 达到 4.34，而 Q-learning、DQN 和 DDQN 仅分别达到 0.66、1.39 和 2.03。持续较高的 Q 值表明 PMR-Dueling DQN 具有近似最佳动作值函数的卓越能力。这会带来更明智的行动选择，进而带来更好的奖励，如奖励图所示。更简单的算法，例如 Q 学习，很难获得高 Q 值，从而限制了它们的性能。总体而言，PMR-Dueling DQN 显然学习到了更好的 Q 函数近似，从而实现了卓越的路径规划。



(**a**) (**b**)

图 13. 四种算法的平均 Q 值比较：(a) 在环境 E-2 中，仅包含静态障碍物；(b) 在环境 E-3 中，同时包含静态和动态障碍物。

6. 结论

这项工作展示了深度强化学习与新颖的网络设计、经验回放和奖励塑造相结合如何能够极大地提高移动机器人路径规划的性能。所提出的 PMR-Dueling DQN 算法使用了这些进展，该算法在网格世界和 Gazebo 模拟环境中都显着优于传统的 Q 学习、DQN 和 DDQN 技术。在网格世界和现实的 Gazebo 模拟环境中进行的评估中强调了 PMR-Dueling DQN 更高的采样效率、稳定性和最终策略质量。与稀疏的二元奖励相比，成形的奖励函数提供了丰富的训练信号，以促进更快的收敛。相反，决斗架构将状态和动作表示分开，以实现更精确的价值学习。优先体验重放进一步补充了这一点，该重放将培训集中在信息最丰富的过渡上。这项研究验证了高保真 Gazebo 环境中的效率，缩小了模拟与现实之间的差距。

虽然模拟环境的结果很有希望，但还需要进一步研究来评估物理机器人平台在现实环境中的性能。合乎逻辑的下一阶段研究涉及将我们提出的 PMR-Dueling DQN 方法的评估扩展到在现实环境中运行的物理机器人。虽然模拟环境提供了宝贵的初始开发空间，但现实世界的动态和不确定性将进一步挑战泛化能力。通过部署到配备用于感知轨迹和障碍物的传感器的平台上，例如带有 LIDAR 和 RGB-D 摄像头的 Turtlebot，我们可以在不同的导航任务中对实时规划性能进行严格的基准测试。穿越办公楼和室外地形的成功将证明该方法对复杂空间的稳健性。过渡到现实世界的自主导航仍然是从模拟发展的最终里程碑。这种解决交互式动态和部分可观察性的努力将增强所提出的技术的能力和成熟度，以实现有影响力的应用。

作者贡献：概念化、D.A.D.和L.Y.；方法论，D.A.D.；监督，L.Y.；软件，D.A.D；调查、D.A.D.、M.L.S.和K.L.；写作——初稿准备，D.A.D.和 M.L.S.；写作——审查和编辑，D.A.D.和K.L.；项目管理，L.Y.所有作者均已阅读并同意稿件的出版版本。

基金资助:该研究得到了国家自然科学基金项目(62373376、61976224和61976088)和湖南省自然科学基金项目(2022JJ30758)的资助。

机构审查委员会声明：不适用。

知情同意声明：不适用。

数据可用性声明：支持本研究结果的数据可根据合理要求从通讯作者处获取，由于隐私原因，这些数据不公开。

利益冲突：作者声明不存在利益冲突 英石。

参考

1. 张海燕;林，W.M.；陈A.X.移动机器人的路径规划：回顾。 *Symmetry* 2018，*10*，450。[交叉引用] 2. Zafar，M.N.； Mohanta, J. 移动机器人路径规划和优化方法：综述。 *Procedia Comput. Sci.* 2018，*133*，141 – 152。 [交叉引用] 3. Tian，S.；雷，S。黄Q。黄A.基于深度强化学习的路径规划算法在移动机器人中的应用。 2022 年文化导向科学技术国际会议 (CoST) 论文集，中国兰州，2022 年 8 月 18 日至 21 日； IEEE：美国纽约州纽约市，2022 年；第 381 – 384 页。 [交叉引用] 4. Patle, B.；巴布，L.G.；潘迪，A.；帕希，D.； Jagadeesh, A. 综述：移动机器人导航的路径规划策略。 *Def. Technol.* 2019，*15*，582 – 606。 [交叉引用] 5. 郭X.；彭，G.；孟，Y。一种改进的 Q 学习算法，用于数字孪生装配系统中的机器人路径规划。 *Int. J. Adv. Manuf. Technol.* 2022 年，*119*，3951 – 3961。 [交叉参考]

6.Bae，H.；金，G。金，J。钱，D。 Lee, S. 使用强化学习的多机器人路径规划方法。 *Appl. Sci.* 2019，*9*，3057。[交叉引用] 7. Holen，M.；萨哈，R。古德温，M.；奥姆林，C.W.；桑德马克，K.E.基于强化学习的自动驾驶汽车的道路检测。第三届国际信息科学与系统会议论文集，英国剑桥，2020 年 3 月 19-22 日；第 67-71 页。 [交叉引用] 8. Xu, J.；田，Y。地图。;罗斯，D.；苏达，S.； Matusik, W. 用于连续机器人控制的预测引导多目标强化学习。第 37 届国际机器学习会议 PMLR 会议记录，虚拟，2020 年 7 月 13-18 日；第 10607 – 10616 页。 9. Mnih，V.；卡武克库奥卢，K.；银，D.；鲁苏，A.A.；维内斯，J.；贝尔马尔，M.G.；格雷夫斯，A.；里德米勒，M.；菲杰兰，A.K.；奥斯特洛夫斯基，G.；等人。通过深度强化学习实现人类水平的控制。 *Nature* 2015，*518*，529 – 533。 [交叉引用] [PubMed] 10. Glimcher，P.W.了解多巴胺和强化学习：多巴胺奖励预测误差假设。 *Proc. Natl. Acad. Sci. USA* 2011 年，*108*，15647 – 15654。 [交叉引用] [PubMed] 11.Manju，S.； Punithavalli, M. 采用奖励函数策略的 Q 学习算法分析。 *Int. J. Comput. Sci. Eng.* 2011，*3*，814 – 820。 12.鲁普雷希特，T.； Wang, Y. 马尔可夫网络物理系统中深度强化学习的调查：常见问题和解决方案。 *Neural Netw.* 2022 年，*153*，13 – 36。 [交叉引用] 13.Kober，J.；巴格内尔，J.A.； Peters, J. 机器人技术中的强化学习：一项调查。 *Int. J. Robot. Res.* 2013，*32*，1238 – 1274。 [交叉引用]14.王Z.；斯考尔，T.；赫塞尔，M.；哈瑟尔特，H.；兰克托，M.； Freitas, N. 用于深度强化学习的决斗网络架构。国际机器学习会议记录 PMLR，美国纽约，2016 年 6 月 20-22 日；第 1995 – 2003 页。 15. Felner, A. 立场文件：Dijkstra 算法与统一成本搜索或反对 Dijkstra 算法的案例。组合搜索国际研讨会论文集，西班牙巴塞罗那，2011 年 7 月 15-16 日；第 2 卷，第 47-51 页。 [交叉引用] 16. Nannicini，G.；戴尔林，D.；利伯蒂，L.； Schultes, D. 双向 A∗ 搜索时间相关的快速路径。实验算法论文集：第七届国际研讨会，WEA 2008，美国马萨诸塞州普罗温斯敦，2008 年 5 月 30 日至 6 月 1 日；施普林格：德国柏林/海德堡，2008 年；第 334 – 346 页。 17. 布尔，V.；奥维马斯，M.H.； Van Der Stappen, A.F. 概率路线图规划者的高斯采样策略。 1999 年 IEEE 国际机器人与自动化会议论文集（目录号 99CH36288C），美国密歇根州底特律，1999 年 5 月 10-15 日：IEEE：美国纽约州纽约，1999 年；第 2 卷，第 1018 – 1023 页。 [CrossRef] 18. LaValle, S. 快速探索随机树：路径规划的新工具。在*Research Report 9811*中；爱荷华州立大学：美国爱荷华州艾姆斯，1998 年。 19. Panov，A.I.；雅科夫列夫，K.S.； Suvorov, R. 使用深度强化学习的网格路径规划：初步结果。 *Procedia Comput. Sci.* 2018，*123*，347 – 353。 [交叉引用]20.张Q.；李，M。王X；张，Y。机器人路径优化中的强化学习。 *J. Softw.* 2012 年，*7*，657 – 662。 [交叉引用]21.阿林，A.； Rabadi, G. 将分布算法估计与 Q 学习集成到 Meta-RaPS 中，以解决 0-1 多维背包问题。 *Comput. Ind. Eng.* 2017 年，*112*，706 – 720。 [交叉引用]22.方，M.； Li, H. 基于成本分析的启发式加速状态回溯 Q 学习。 *Int. J. Pattern Recognit. Artif. Intell.* 2013，*35*，838 – 844。 23.王YH；李，T.H.S.； Lin, C.J. 向后 Q 学习：Sarsa 算法和 Q 学习的结合。 *Eng. Appl. Artif. Intell.* 2013 年，*26*，2184 – 2193。 [交叉引用] 24. Duguleana，M.； Mogan, G. 基于神经网络的移动机器人避障强化学习。 *Expert Syst. Appl.* 2016 年，*62*，104 – 115。 [交叉引用] 25. Rakshit，P.；科纳尔，A.；博米克，P.；戈斯瓦米，I.；达斯，S。贾恩，L.C.；纳加尔，A.K.使用差分进化和 Q 学习实现自适应模因算法：多机器人路径规划的案例研究。 *IEEE Trans. Syst. Man Cybern. Syst.* 2013，*43*，814 – 831。 [交叉引用] 26. 卡卢乔，I.；德保拉，M.；王，S。佩蒂洛特，Y.；阿科斯塔，G.G.使用深度强化学习的自主水下航行器的自适应低级控制。 *Robot. Auton. Syst.* 2018，*107*，71 – 86。 [交叉引用] 27. Kayakoku，H.；古泽尔，硕士；博斯坦奇，E.；梅德尼，I.T.； Mishra, D. 基于深度 Q 学习的 RoboCode 平台的新颖行为策略。 *Complexity* 2021，*2021*，9963018。[交叉引用] 28. 高，X.；罗，H.；宁，B.；赵，F。鲍，L.；龚，Y。肖，Y。 Jiang, J. RL-AKF：一种基于强化学习的地面车辆自适应卡尔曼滤波导航算法。 *Remote. Sens.* 2020，*12*，1704。[交叉引用] 29。你，C.；卢，J。菲列夫，D.； Tsiotras, P. 使用强化学习和深度逆强化学习对自动驾驶车辆进行高级规划。 *Robot. Auton. Syst.* 2019，*114*，1-18。 [交叉引用] 30. Maeda，Y.；渡边，T.； Moriyama, Y. 基于视图的编程和机器人操作的强化学习。 2011 年 IEEE 国际研讨会论文集

33. 埃斯科巴-纳兰霍，J.；凯萨，G.；阿亚拉，P.；乔丹，E。加西亚，加利福尼亚州；加西亚，M.V.机器人自主导航：使用 DQN 进行优化。 *Appl. Sci.* 2023，*13*，7202。[交叉引用] 34. Van Hasselt，H.；格斯，A.； Silver, D. 使用双 q 学习的深度强化学习。 AAAI 人工智能会议记录，美国亚利桑那州凤凰城，2016 年 2 月 12 日至 17 日；第 30 卷。[交叉引用] 35。Quan，L.；严，Y。 Fei, Z. 具有探索性噪声的深度循环 Q 网络。 *Chin. J. Comput.* 2019，*42*，1588 – 1604。 36.夏Z.； Qin，J。深度 Q 网络的改进算法。 *J. Comput. Appl. Res.* 2019，*36*，3661 – 3665。 37.金，K.S.；金，D.E.； Lee，J.M. 基于平滑驾驶的自主导航深度学习。 2018 年 IEEE/ASME 国际高级智能机电一体化会议 (AIM) 会议记录，新西兰奥克兰，2018 年 7 月 9-12 日； IEEE：美国纽约州纽约市，2018 年；第 616 – 621 页。 [交叉引用] 38.阮X.；任，D.；朱X。基于深度强化学习的移动机器人导航。 2019 年中国控制与决策会议 (CCDC) 论文集，中国南昌，2019 年 6 月 3-5 日； IEEE：美国纽约州纽约市，2019 年，第 6174 – 6178 页。 [交叉引用] 39.陈，G.；潘，L.；陈，Y。徐，P。王，Z。吴，P。吉，J。 Chen, X. 基于地图的移动机器人导航避障深度强化学习。 *SN Comput. Sci.* 2021，*2*，417。[交叉引用] 40.周，F.Y.；金，L.P.； Dong, J. 卷积神经网络综述。 *J. Comput.* 2017，*40*，1229 – 1251。 41.斯考尔，T.；全，J.；安东诺格鲁，I.； Silver, D. 优先体验重播。 *arXiv* 2015，arXiv：1511.05952。 [交叉参考]

Disclaimer/Publisher's Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.